

МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
**«САРАТОВСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИМЕНИ Н. Г. ЧЕРНЫШЕВСКОГО»**

УТВЕРЖДАЮ

Зав.кафедрой,

доцент, к. ф.-м. н.

_____ С. В. Миронов

ОТЧЕТ О ПРАКТИКЕ

студентки 3 курса 351 группы факультета КНиИТ

Винник Маргариты Дмитриевны

вид практики: учебная

кафедра: математической кибернетики и компьютерных наук

курс: 3

семестр: 6

продолжительность: 2 нед., с 08.05.2026 г. по 26.05.2026 г.

Руководитель практики от университета,

доцент, к. ф.-м. н.

М. И. Сафрончик

Руководитель практики от организации (учреждения, предприятия),

доцент, к. ф.-м. н.

М. И. Сафрончик

Тема практики: «Модули в языке Python»

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	4
1 Теоретические сведения	6
1.1 Библиотека NumPy	6
1.2 Библиотека Pandas	9
1.3 Библиотека Scikit-learn	9
1.4 Библиотека Matplotlib / Seaborn	9
ЗАКЛЮЧЕНИЕ	9

ВВЕДЕНИЕ

Сам по себе «чистый» Python, безусловно, удобен, но для сложных численных расчётов, анализа данных (Data Science) и машинного обучения (Machine learning) его встроенных возможностей недостаточно. Ключевая причина, почему Python стал лидером в области анализа данных и искусственного интеллекта — это его способность легко интегрироваться с кодом на других языках.

Ядро интерпретатора CPython (самая распространённая реализация Python) написано на C и C++.

Этот фундамент обеспечивает эффективное управление памятью, работу с объектами и выполнение базовых операций. Многие встроенные функции (print, len) и модули стандартной библиотеки (math, sys) также реализованы на C для достижения максимальной производительности.

Однако настоящая мощь раскрывается в сторонних библиотеках. Такие «тяжеловесы», как NumPy для векторных вычислений, Pandas для анализа табличных данных, а также фреймворки глубокого обучения PyTorch и TensorFlow, являются модулями расширения на C/C++.

Эта же философия расширения лежит в основе библиотек для классического машинного обучения и визуализации. Библиотека scikit-learn, построенная поверх NumPy и SciPy, содержит оптимизированные реализации алгоритмов на C/C++ (например, для вычисления расстояний, построения деревьев решений), предоставляя при этом единый Python-интерфейс.

Для задач визуализации данных фундаментальная библиотека matplotlib наследует возможности векторной графики из C++-библиотек (в частности, Anti-Grain Geometry), а высокоуровневая надстройка seaborn позволяет создавать сложные статистические графики минимальным количеством кода, полагаясь на ту же C/C++-основу.

Таким образом, каждая из ключевых библиотек экосистемы Data Science следует принципу "Python-обёртка над высокопроизводительным ядром".

В этом контексте Python выступает в роли высокоуровневого «клея»: он предоставляет удобный и гибкий интерфейс для сборки и управления высокопроизводительными вычислительными ядрами, написанными на компилируемых языках. Такой подход сочетает простоту разработки со скоростью выполнения кода, что критически важно для задач анализа данных и машинного обучения.

Важно отметить, что все эти библиотеки спроектированы для бесшовной интеграции друг с другом. NumPy обеспечивает единый формат данных (ndarray), который понимают Pandas, SciPy, Scikit-learn и Matplotlib. Pandas, в свою очередь, построен поверх NumPy и добавляет удобную работу с табличными данными и временными рядами. Scikit-learn принимает на вход как NumPy-массивы, так и Pandas-DataFrame. Seaborn изначально задуман для работы с DataFrame и интегрируется с Matplotlib для тонкой настройки графиков. Эта взаимная совместимость создаёт цельную экосистему, где каждая библиотека усиливает возможности другой, оставаясь при этом частью единого "языкового клея"— Python.

1 Теоретические сведения

О чём собственно я буду рассказывать:

В этой работе я рассмотрю несколько основных библиотек-инструментов (модулями на самом деле их уже давно не называют из-за их объёмности и сложности) для специалиста анализа данных и инженера машинного обучения.

Вот он - так называемый джентельменский набор, который я постепенно рассмотрю подробнее, объясню что для чего нужно и как использовать:

1. NumPy (Numerical Python);
2. Pandas;
3. Scikit-learn;
4. Matplotlib / Seaborn;

1.1 Библиотека NumPy

По классике, начнём с NumPy, но немного издаleка, а именно с типа данных, который имеют объекты, созданные при помощи этой библиотекой. Тип данных **ndarray** библиотеки NumPy представляет собой многомерный массив элементов одного типа. Элементы массива NumPy индексируются кортежем целых неотрицательных чисел.

Массивы NumPy похожи на списки (list) Python или на массивы из встроенного модуля **array**, которые используются для создания плотных массивов данных одного типа. Однако массивы NumPy требуют меньше памяти и обычно работают быстрее, поскольку используют оптимизированный, прекомпилированный код на языке C.

В то время как объект **array** в языке Python обеспечивает только эффективное хранение данных в формате массива, библиотека NumPy добавляет ещё и возможность выполнения эффективных операций с этими данными (будут рассмотрены дальше).

Ключевая особенность — поддержка поэлементных операций. Это позволяет выполнять основные арифметические вычисления сразу над всем массивом, делая код компактным и удобочитаемым. Также доступна поэлементная операция над двумя массивами одинаковой размерности, в результате которой получается другой массив той же размерности, где каждый элемент i, j является результатом вычисления, выполненного над элементами i, j двух исходных массивов.

Таблица 1.1 – Сравнение типов данных для хранения массивов в Python

Характеристика	Встроенный list	Встроенный array из модуля array	ndarray из NumPy
Типы данных	Любые, различные	Однородный (один тип)	Однородный (один тип)
Размер	Динамический	Фиксированный	Фиксированный (при создании)
Производительность	Низкая	Средняя	Высокая (благодаря C/C++ ядру)
Функциональность	Базовая (добавление, удаление)	Очень базовая	Огромная (векторизация, линейная алгебра, статистика)
Потребление памяти	Большое	Маленькое	Маленькое (плотное хранение)

Прежде чем начать работать с этой библиотекой, её нужно установить, т.к. она не является встроенным модулем, как, например, модули `math`, `array` и другие стандартные. Установка NumPy происходит по команде в терминале (VS Code, PyCharm): **`pip install numpy`**. Импортируем пакет NumPy под псевдонимом **`np`** (это уже в рабочем файле `.py`):

```
import numpy as np
```

Дальше в примерах кода буду опускать строку с импортом.

Итак, способы создания массивов NumPy (буду сразу приводить примеры кода):

- Массив NumPy можно создать из данных одного или нескольких списков. Для создания массивов из списков можно воспользоваться функцией **`np.array`**.

```
my_arr = np.array([1, 4, 2, 5, 3])
```

Если мы захотим вывести такой массив, то увидим в консоле следующее: **`[1 4 2 5 3]`**.

С помощью функции **`type()`** можем убедиться, что тип созданного объекта **`ndarray`**. В консоли увидим: **`<class 'numpy.ndarray'>`**.

Сразу рассмотрим пример создания многомерного массива из нескольких "листов":

Для каждого сотрудника компании существует список выплат базового оклада за последние три месяца. Чтобы собрать всю информацию о зарплате в одну структуру данных, можно использовать следующий код:

```
jeff_salary = [2700, 3000, 3000]
nick_salary = [2600, 2800, 2800]
tom_salary = [2300, 2500, 2500]
base_salary = np.array([jeff_salary, nick_salary, tom_salary])
print(base_salary)
```

В консоли это будет выглядеть так:

```
[[2700 3000 3000]
 [2600 2800 2800]
 [2300 2500 2500]]
```

- Явное описание многомерного массива (матрицы). Вот один из способов инициализации многомерного массива с помощью списка списков:

```
my_arr = np.array([range(i, i + 3) for i in [2, 4, 6]])
print(my_arr)
```

В консоли это будет выглядеть так:

```
[[2 3 4]
 [4 5 6]
 [6 7 8]]
```

Простейшие операции над массивами NumPy:

- получение атрибутов массивов — определение размера, формы, объема занимаемой памяти и типов данных массивов;
- индексация массивов — получение и изменение значений отдельных элементов массивов;
- получение срезов массивов — получение и изменение значений подмассивов внутри большого массива;
- изменение формы массивов — изменение формы заданного массива;
- объединение и разбиение массивов — объединение нескольких массивов в один и разделение одного массива на несколько.

1.2 Библиотека Pandas

1.3 Библиотека Scikit-learn

1.4 Библиотека Matplotlib / Seaborn

ЗАКЛЮЧЕНИЕ